SITE AUTOMATION

AAP with the Keycloak (CAC) login

AAP behind STIG Manager's nginx, a Keycloak realm of its own, the people from Active Directory

Site Automation - set up, test, roll back | site-automation 1.64.0 | November 2026

What this is. People open AAP's address, pick their card's certificate and are in AAP, in the right organization and team. This guide sets it up in order: what to check first, the nginx blocks to paste, AAP's proxy settings, the Keycloak realm (made by a script), the AAP login method, the tests and the roll back - and how to build the same in an air-gapped enclave.

How to read it
Parts 1 to 7 in order; each step is one action. Rendered from `docs/AAP_KEYCLOAK.md` in the site-automation repository (version 1.64.0); copy commands and settings from the repository, not from the PDF.

Host names and addresses in this guide are placeholders. Copy YAML from the repository files, not from the PDF.

Contents

AAP with the Keycloak (CAC) login 3

1. Check first . . . 4
2. nginx: the blocks to paste . . . 6
3. AAP: tell the gateway it is behind a proxy . . . 10
4. Keycloak: a realm for AAP, made by a script . . . 11
5. AAP: the login method and the maps . . . 19
6. Test . . . 22
7. Roll back . . . 27

Building it in an air-gapped enclave . . . 27
The three scripts . . . 30
Where each step comes from . . . 34
Not in Red Hat's documents . . . 36
What was tried . . . 36

AAP with the Keycloak (CAC) login

People open `https://aap.yoursite.mil`, click **Keycloak**, pick their card's certificate, and are in AAP - in the right organization and team. The local `admin` login stays, for the day Keycloak is down.

This is a checklist. Do the parts in order. Each step is one action, with the text to type and what you should see.

- **For:** AAP 2.5 or later, the **containerized** install (the platform gateway runs in podman).
- **nginx:** the one STIG Manager already has (the container `stigman-nginx`). It gets **more blocks, pasted by hand**: one for AAP, the card's way to Keycloak, and - when you want it - the report website. No new container, no new image. Nothing in this repository writes to STIG Manager's nginx, containers or database.
- **Keycloak:** the one you already run. AAP gets **a realm of its own** (`aap`), made by a script. STIG Manager's realm is not touched: the script refuses every request to another realm.
- **Three scripts, one settings file:**

Script	What it does	Changes something?
`tools/aap-nginx-blocks.py`	writes the nginx blocks and the steps to `./out`, for you to paste (part 2)	only the two files in `./out`
`tools/aap-keycloak-realm.py`	makes the realm `aap` in Keycloak (part 4)	only with `--apply`, only in that realm
`tools/aap-keycloak-check.sh`	prints PASS or FAIL for each link of the chain (part 6)	never

- **The people** come from Active Directory: the members of one group, read-only (part 4, "People from Active Directory"). A list or Keycloak's console work too.
- **You built the realm by hand already:** part 4, "If you already set it up by hand". The script adds what is missing and changes nothing that is there.
- **Another network, or one with no internet:** "Building it in an air-gapped enclave", after part 7.

What was tried, and what was not. The realm script, the nginx blocks and a card login through them were tried on a real Keycloak 26.0.6 behind a real nginx, with a test CA and test cards. The people from Active Directory were tried against a test domain controller (see "What was tried" at the end). **Not tried:** a real AAP, a real CAC, your own directory. The AAP steps come from Red Hat's documents and from the source code of AAP's login. The steps that are **not** in Red Hat's documents are marked *(not in Red Hat's documents)* and listed at the end. Do it first on a test AAP if you have one.

The names used here - put yours in their place everywhere:

Here	What it is
`aap.yoursite.mil`	the new name people will type for AAP
`aap01.yoursite.mil`	the AAP host itself (where the gateway runs)
`stigman.yoursite.mil`	the STIG Manager host (where nginx runs)
`keycloak.yoursite.mil`	Keycloak's address, as STIG Manager's login uses it

Here	What it is
`aap`	the new Keycloak realm for AAP (`KEYCLOAK_REALM`), and the client in it (`KEYCLOAK_CLIENT_ID`)
`stigman`	the realm STIG Manager uses - never the one this guide works in
`aap-keycloak.env`	your settings file (step 1.8). One per network, never in Git.
`<nginx>`	the nginx container's name (step 1.3)
`<keycloak>`	the Keycloak container's name (step 1.4)
`<the file>`	nginx's config file on the STIG Manager host (step 1.3)

1. Check first

Nothing is changed in this part.

1. **AAP is 2.5 or later.** In AAP: the **?** at the top right > **About**. You should see version 2.5 or 2.6.
2. **AAP is containerized, and the gateway answers.** On the AAP host, as the user that installed AAP (not root):

```shell
podman ps --format "{{.Names}}" | grep automation-gateway
```

You should see two lines: `automation-gateway` and `automation-gateway-proxy`. Nothing: this is not a containerized install - stop, this guide is not for it.

Then find the gateway's port. In the installer's `inventory` file, look for `envoy_https_port`. No such line: the port is 443. Check it answers, from the STIG Manager host:

```shell
curl -sk -o /dev/null -w '%{http_code}\n' https://aap01.yoursite.mil:443/api/gateway/v1/ping/
```

You should see `200`. (`-k` only here: the gateway's own certificate is often not from your CA.) `000`: a firewall between the two hosts, or the wrong port - fix that first.

3. **The nginx container and its file.** On the STIG Manager host (`sudo` throughout: its containers are root's):

```shell
sudo podman ps --format "{{.Names}}  {{.Networks}}  {{.Ports}}"
```

Note the **nginx container's name** (the one with `443` in its ports) and the **port before** `->443`. That port is the one people use: when it is not 443, the address is `https://aap.yoursite.mil:<that port>` everywhere below. Then:

```shell
sudo podman inspect <nginx> --format '{{range .Mounts}}{{.Source}} ->
  {{.Destination}}{{println}}{{end}}'
```

You should see two lines or more. The one ending in `.conf` is **the file** (its left side, on the host). The one ending in `/etc/nginx/tls` is **the certificate folder** (its left side). Then read three lines of the file:

```shell
sudo grep -n -E 'listen|ssl_client_certificate|ssl_verify_client|proxy_pass' <the file>
```

Note the `listen` line of STIG Manager's block, the file its `ssl_client_certificate` line names (the CAs of people's cards), and whether a `proxy_pass` line names Keycloak - then Keycloak is behind this nginx already.

4. **Keycloak answers, and it reads the card.** From the AAP host, with the name of STIG Manager's realm:

```shell
curl -s https://keycloak.yoursite.mil/realms/stigman/.well-known/openid-configuration | head -c
  200
```

You should see text that starts with `{"issuer":"https://keycloak.yoursite.mil/realms/stigman"`. Three things can go wrong here, and each must be fixed before you go on:

- *Nothing*, or a certificate error: the AAP host does not reach Keycloak or does not trust your CA.
- `404`: an old Keycloak keeps its realms under `/auth` - use `https://keycloak.yoursite.mil/auth` as Keycloak's address everywhere below.
- *No required SSL certificate was sent*: Keycloak's address asks every caller for a card. AAP calls Keycloak itself, with no card, to finish a login. Step 2.6 changes that to `ssl_verify_client optional` (the card is then still asked from people, and checked by Keycloak).

Then, on the Keycloak host, see how Keycloak was started:

```shell
sudo podman ps --format "{{.Names}}  {{.Image}}" | grep -i -E 'keycloak|auth'
sudo podman inspect <keycloak> --format '{{.Config.Cmd}}{{println}}{{range .Config.Env}}{{println
  .}}{{end}}' | grep -i -E 'x509|proxy|truststore'
```

You should see a line with `x509cert-lookup` and `nginx` in it (as `--spi-x509cert-lookup-provider=nginx` or `KC_SPI_X509CERT_LOOKUP_PROVIDER=nginx`). Then Keycloak already reads the card from nginx - STIG Manager's card login works that way - and step 4.1 is done. No such line: step 4.1 adds it.

5. **The new name.** Ask for a DNS record: `aap.yoursite.mil` points at the **STIG Manager host's** address (where nginx is), not at the AAP host. Check, from your workstation and from the AAP host:

```shell
getent hosts aap.yoursite.mil
```

You should see the STIG Manager host's address, on both.

6. **A certificate for the new name.** Ask your PKI for a server certificate for `aap.yoursite.mil` (as you did for STIG Manager's). You need the certificate with its chain, and the key, as two PEM files.
7. **The report website's folder** - only when nginx should serve it too. It is the folder the report jobs write to (`report_web_dest`). nginx can only serve a folder **the STIG Manager host has**: a local folder, or a share mounted on that host. Check, on the STIG Manager host:

```shell
ls /srv/site-reports
```

You should see `index.html` and one folder per report (after a first report has run with `report_web_dest`). The folder is on another machine: mount that share on the STIG Manager host first, or see "no restart" in step 2.5.

8. **The settings file.** On your workstation, in this repository's folder:

```shell
cp tools/aap-keycloak.env.example aap-keycloak.env
nano aap-keycloak.env
```

Put your names in: `AAP_URL`, `AAP_UPSTREAM` (step 1.2), `KEYCLOAK_URL`, `STIGMAN_URL`, `NGINX_CONTAINER` and `NGINX_LISTEN` (step 1.3), and STIG Manager's realm in `KEYCLOAK_PROTECTED_REALMS`. For the report website: `REPORT_WEB_FOLDER` (step 1.7). No password goes in this file. The three scripts read it with `--settings`. The file is yours: Git passes over it (`.gitignore` holds `aap-keycloak*.env`).

Why its own name and not `https://stigman.yoursite.mil/aap/`: AAP's web page asks for its API at fixed addresses that start at `/` (`/api/gateway/...`, `/api/controller/...`, the websocket). It cannot live under a sub-path. Red Hat's documents give no way to set one. *(Not in Red Hat's documents: they do not say it in one sentence. It follows from the page's source code and from the only setting there is, `gateway_main_url`, a whole address.)*

2. nginx: the blocks to paste

On the STIG Manager host. The blocks are made by a script from your settings. You paste them by hand, with a copy kept first. No script writes nginx's file.

Block	What it is	Where it goes
A - AAP	a `server` block for `aap.yoursite.mil`	at the end of the file
B - Keycloak's card	the line that hands the card's certificate to Keycloak (`KEYCLOAK_NGINX=lines`), or a whole `server` block for Keycloak's name (`KEYCLOAK_NGINX=server`)	inside the Keycloak location nginx has, or at the end of the file
C - Reports	`location /reports/`, the report website's folder (only with `REPORT_WEB_FOLDER`)	inside STIG Manager's own `server` block

The new realm needs **no new address**: it lives at Keycloak's own address, under `/realms/aap`. Block B only makes sure the card reaches Keycloak there.

1. **Make the blocks.** On your workstation, in this repository's folder:

```shell
tools/aap-nginx-blocks.py --settings aap-keycloak.env
```

You should see two `wrote` lines: `out/aap-nginx-blocks.conf` (the blocks) and `out/aap-nginx-steps.txt` (the steps below, with your names in them). Copy both to the STIG Manager host. The script wrote nothing else, and it refuses a folder where nginx's own files live.

2. **Keep a copy** of nginx's file:

```shell
sudo cp <the file> <the file>.before-aap
```

3. **Check this host reaches the gateway** (the same line as step 1.2):

```shell
curl -sk -o /dev/null -w '%{http_code}\n' https://aap01.yoursite.mil:443/api/gateway/v1/ping/
```

You should see `200`.

4. **Put the certificate in the certificate folder** (step 1.3). Use `cp`, not `mv`, so the files get the folder's SELinux label:

```shell
sudo cp aap-fullchain.pem <the certificate folder>/aap-fullchain.pem
sudo cp aap-key.pem <the certificate folder>/aap-key.pem
sudo chmod --reference=<the certificate folder>/key.pem <the certificate folder>/aap-key.pem
sudo podman exec <nginx> ls /etc/nginx/tls
```

You should see `aap-fullchain.pem` and `aap-key.pem` in the list. (STIG Manager's certificate already names `aap.yoursite.mil` as well? Then set `AAP_TLS_CERT` and `AAP_TLS_KEY` to its two file names and skip this step.)

5. **The report folder's mount - this restarts STIG Manager's nginx.** Only for block C. Skip it without `REPORT_WEB_FOLDER`.

People lose STIG Manager (and AAP, once it is behind this nginx) for some seconds. A running container cannot be given a folder: the mount is added to how the container starts, and the container is started again. Do it in a quiet hour. People's logins survive it.

- nginx runs from a **systemd unit** (`/etc/containers/systemd/stigman-nginx.container`): add this line under `[Container]`, below the other `Volume=` lines:

```
Volume=/srv/site-reports:/usr/share/nginx/site-reports:ro,z
```

```shell
sudo systemctl daemon-reload
sudo systemctl restart stigman-nginx.service
```

- nginx runs from a **compose file**: add this line under the nginx service's `volumes:`, then start that one service again, in the compose file's folder:

```yaml
- /srv/site-reports:/usr/share/nginx/site-reports:ro,z
```

```shell
sudo podman-compose up -d --force-recreate --no-deps <the nginx service>
```

Check the container sees the folder:

```shell
sudo podman exec <nginx> ls /usr/share/nginx/site-reports
```

You should see the report website's files.

- `ro`: nginx only reads the folder. The jobs write it, on the host.

- `,z` lets the container read the folder under SELinux. On a folder that is an NFS or CIFS mount, leave `,z` out (write `:ro`) and allow it once: `sudo setsebool -P virt_use_nfs 1` (or `virt_use_samba 1`).
- **No restart** is possible only when another web server already serves the folder. Set `REPORTS_UPSTREAM` to it (for example `http://web01.yoursite.mil:8080`) and take `REPORT_WEB_FOLDER` out, then run step 2.1 again. Block C then passes requests on to that server, behind STIG Manager's card check, and needs only the reload of step 2.9. There is no other way: nginx cannot serve a folder its container does not see.

6. **Paste the blocks** into `<the file>`, each where its `# Where:` line says:
   - **A** at the very **end** of the file, after the last `}` - below STIG Manager's own `server` block, never inside it and never above it (the first block in the file answers names nginx does not know: that must stay STIG Manager).
   - **B** (`lines`) inside the `location` that already passes requests on to Keycloak, beside its other `proxy_set_header` lines. A line that is already there: leave it. Then look at the `ssl_verify_client` line of the `server` block around it. It must say `optional`. It says `on`: change that one word. People are still asked for the card, and Keycloak checks it; AAP itself, which calls Keycloak with no card, is let through.
   - **C** inside STIG Manager's own `server` block, above its `location / {` line.

Edit the file **in place**: `sudo nano <the file>` does; in vim first `:set backupcopy=yes`. The container sees that one file: an editor that saves a new file in its place leaves nginx reading the old one.

7. **Check the container sees the lines.** This must print `1`:

```shell
sudo podman exec <nginx> grep -c 'server_name aap.yoursite.mil' /etc/nginx/conf.d/default.conf
```

(Use the right side of step 1.3's `.conf` line when it is not `default.conf`: `NGINX_CONF`.) A `0`: the editor saved a new file. Put the copy back (`sudo cp <the file>.before-aap <the file>`) and edit again, in place.

8. **Test it:**

```shell
sudo podman exec <nginx> nginx -t
```

You should see *syntax is ok* and *test is successful*. Anything else: put the copy back, and nothing has changed. (*host not found in upstream*: the container does not resolve the AAP host's name - use its address in `AAP_UPSTREAM` and make the blocks again.)

9. **Load it:**

```shell
sudo podman exec <nginx> nginx -s reload
```

nginx reloads without dropping anyone. STIG Manager keeps working, and its sessions too.

10. **Check the names**, from your workstation:

```shell
curl -s -o /dev/null -w '%{http_code}\n' https://aap.yoursite.mil/api/gateway/v1/ping/
```

You should see `200`. Then open STIG Manager in the browser: it works as before, and still asks for the card. With block C: open `https://stigman.yoursite.mil/reports/` with your card. You should see the report

website's first page. Then set `report_web_url: https://stigman.yoursite.mil/reports` in the report jobs, so the emails link to it.

What block A looks like, and what each group of lines is for:

```
server {
    listen 443 ssl;
    server_name aap.yoursite.mil;
    ssl_verify_client off;                # no card asked here: Keycloak checks the card
    client_max_body_size 200m;
    location / {
        proxy_pass https://aap01.yoursite.mil:443;
        proxy_http_version 1.1;
        proxy_set_header Host $http_host;
        proxy_set_header X-Forwarded-Host $http_host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        ...
    }
}
```

Lines	Why
`map` and the `Upgrade` / `Connection` headers, `proxy_http_version 1.1`	the websocket: a job's output while it runs. Red Hat: the proxy must talk HTTP/1.1 to the gateway, HTTP/2 is not supported.
`Host` and `X-Forwarded-Host`	AAP builds the address Keycloak sends people back to from this. `$http_host` keeps the port.
`X-Forwarded-Proto https`	AAP knows the browser used https.
`X-Forwarded-For`, `X-Real-IP`	AAP's logs show the person's address, not nginx's.
`proxy_read_timeout`, `proxy_send_timeout`	a quiet websocket and a slow API call are not cut after 60 seconds.
`client_max_body_size`	nginx's default is 1m: a project or collection upload would fail with *413*.
`ssl_verify_client off`	see below.
B: `proxy_set_header ssl-client-cert $ssl_client_escaped_cert;`	nginx ends TLS, so Keycloak never sees the card itself. This line hands it the card's certificate. nginx sets the header on every request, so a caller cannot send a made-up one through nginx.
C: `alias`, `limit_except GET HEAD`	the folder is served as it is, read-only. It sits inside STIG Manager's block: the same certificate, the same card check. The pages need no change: they already work under a strict content security policy.

Keep `ssl_verify_client off` on the AAP block. STIG Manager's block asks every caller for a card (mTLS). AAP's must not. People's cards are still checked: Keycloak does it at the login. A card asked here as well would break everything that talks to AAP without a person:

- scripts and tokens: `curl`, the `ansible.platform` and `ansible.controller` collections, and this repository's own jobs that read AAP's API (the strip of past runs, the AAP health check, the template

audit);
- AAP's own parts and its execution nodes, once `gateway_main_url` is this name (part 3);
- webhooks from Git or ServiceNow, and provisioning callbacks from new hosts;
- Event-Driven Ansible's event streams.

Keycloak's own port must be reachable from nginx only. Keycloak believes the `ssl-client-cert` header. A certificate is public: anyone who reaches Keycloak's own port directly (8080 or 8443, not through nginx) can send another person's certificate in that header and be logged in as them. This was tried, and it works. Keep Keycloak's port unpublished (the containers' own network) or closed by the firewall to everything but nginx. This is true for STIG Manager's card login too, today.

If STIG Manager's nginx file or unit is written by a job - this repository's *STIG Manager - deploy* writes both on each run (*Managed by Ansible*) - that run takes the blocks and the `Volume=` line out again. Paste them again after it (steps 5 to 9). Keep `out/aap-nginx-blocks.conf` beside the copy for that day.

3. AAP: tell the gateway it is behind a proxy

AAP now answers at the new name. Two settings make it sure of that name. Do them now, before Keycloak.

1. **The address, in the installer's inventory.** On the AAP host, in the installer folder, open `inventory` and add under `[all:vars]`:

```
gateway_main_url=https://aap.yoursite.mil
custom_ca_cert=/home/aap/your-ca-chain.pem
```

`custom_ca_cert` is your CA chain as a PEM file on that host: AAP then trusts nginx's and Keycloak's certificates. It is already in the inventory? Leave it.

Know what this does: AAP's own parts (and execution nodes) then reach the gateway at this address - through STIG Manager's nginx. Step 1.5's check from the AAP host must have passed.

2. **Run the installer again**, the same way AAP was installed, as the same user:

```shell
ansible-playbook -i inventory ansible.containerized_installer.install
```

(Add the options you used at install, such as `-e @vault.yml --ask-vault-pass`.) You should see it end with `failed=0`. It restarts AAP's containers: do it in a quiet hour.

3. **The trusted origin.** Log in at `https://aap.yoursite.mil` as `admin` (the local login). Go to **Settings > Platform gateway > Edit**. In **Csrf trusted origins** add:

```
https://aap.yoursite.mil
```

Save. (AAP 2.6 has this field. On 2.5 there is no such field: put this line in the inventory instead and run the installer again - *not in Red Hat's documents for the containerized install*: `gateway_extra_settings=[{"setting": "CSRF_TRUSTED_ORIGINS", "value": ["https://aap.yoursite.mil"]}]`)

4. **Whose address AAP logs** - optional. **Settings > Automation Execution > System > Edit**:
   - **Proxy IP Allowed List:** the address nginx's requests come from, as the AAP host sees it (the STIG Manager host's address), for example `["192.0.2.10"]`.
   - **Remote Host Headers:**

```
["HTTP_X_FORWARDED_FOR", "REMOTE_ADDR", "REMOTE_HOST"]
```

Fill the first before the second. Red Hat warns: with `HTTP_X_FORWARDED_FOR` and an empty allowed list, anyone who reaches AAP directly can fake their address.

5. **Check:** log out, and log in again at `https://aap.yoursite.mil` as `admin`. You should be in, with no *CSRF* or *Origin checking failed* message. Open a job's output: it shows.

4. Keycloak: a realm for AAP, made by a script

AAP gets a realm of its own, `aap`, in the Keycloak you already run. `tools/aap-keycloak-realm.py` makes it.

- Without `--apply` it changes nothing. It prints every change it would make.
- With `--apply` it makes them. A second `--apply` finds nothing to change. It never deletes anything.
- **It never touches another realm.** Every request goes to `/admin/realms/aap...`. The only others are the admin login and the one request that makes the realm. A request to any other address is refused inside the script before it is sent, and the last line of each run counts them: `to another realm: 0`.
- One thing Keycloak does by itself: when any realm is made, Keycloak adds a client named `aap-realm` to its own `master` realm (that is how `master`'s admins manage the new realm). The script does not ask for it and cannot prevent it. STIG Manager's realm does not change.
- Python 3.6 or later, nothing to install.

1. **Keycloak reads the card from nginx - a change you make by hand.** Skip this step when step 1.4 showed the `x509cert-lookup` line.

nginx ends TLS, so Keycloak must be told to take the card's certificate from the header nginx sends (block B). These are **start options of the Keycloak container you already run**. No script here sets them. Adding them **restarts Keycloak**: nobody can log in to STIG Manager for that minute. Do it in a quiet hour.

Start option (after start)	The same as an environment variable	Why
`--spi-x509cert-lookup-provider=nginx`	`KC_SPI_X509CERT_LOOKUP_PROVIDER=nginx`	read the card from nginx's header
`--spi-x509cert-lookup-nginx-ssl-client-cert=ssl-client-cert`	`KC_SPI_X509CERT_LOOKUP_NGINX_SSL_CLIENT_CERT=ssl-client-cert`	the header's name, as in block B
`--spi-x509cert-lookup-nginx-ssl-cert-chain-prefix=CERT_CHAIN`	`KC_SPI_X509CERT_LOOKUP_NGINX_SSL_CERT_CHAIN_PREFIX=CERT_CHAIN`	Keycloak's own default, said out loud
`--spi-x509cert-lookup-nginx-certificate-chain-length=2`	`KC_SPI_X509CERT_LOOKUP_NGINX_CERTIFICATE_CHAIN_LENGTH=2`	how many CA certificates are above a card (a DoD card: 2)
`--truststore-paths=/opt/keycloak/conf/card-cas.pem`	`KC_TRUSTSTORE_PATHS=/opt/keycloak/conf/card-cas.pem`	the CAs of people's cards (PEM), so Keycloak rebuilds the card's chain. The file is the one of nginx's `ssl_client_certificate` line, mounted into the Keycloak container.

Start option (after start)	The same as an environment variable	Why
`--proxy-headers=xforwarded`	`KC_PROXY_HEADERS=xforwarded`	Keycloak is behind a proxy. It is there already when STIG Manager's login works.

Where they go:

- a **compose file**: under the Keycloak service, as `environment:` lines (`KC_...: value`) or added to its `command:`; then `sudo podman-compose up -d --force-recreate --no-deps <the keycloak service>`;
- a **systemd unit** (quadlet): as `Environment=KC_...=value` lines under `[Container]`; then `sudo systemctl daemon-reload` and `sudo systemctl restart <the unit>`.

Check, after the restart:

```shell
sudo podman inspect <keycloak> --format '{{.Config.Cmd}}{{println}}{{range .Config.Env}}{{println
  .}}{{end}}' | grep -i x509
```

You should see the `x509cert-lookup` lines. Then log in to STIG Manager with your card: it works as before.

- Without the truststore the login still works, and Keycloak's log warns at each one (*No issuer certificate ... Impossible to rebuild end user cert chain*). `X509_OCSP` and `X509_CRL` need the chain: give the truststore.
- These options are for the whole Keycloak, every realm. They change nothing for a realm with no card flow.
- With people from Active Directory, the directory's CA chain goes in `--truststore-paths` too, after a comma ("People from Active Directory", step 3). One restart does both.

2. **How the script reaches Keycloak.** It picks by itself (`--via auto`), in this order:

Way	Where you run it	What it needs
`rest`	any machine that reaches Keycloak's admin address	`KEYCLOAK_ADMIN_URL` when the admin address is not `KEYCLOAK_URL`
`kcadm`	the Keycloak host, with `sudo`	`KEYCLOAK_CONTAINER`: it runs `kcadm.sh` inside that container

Keycloak's admin address is closed from your workstation (a common hardening): run the script on the Keycloak host, where `kcadm` is used. Both ways make the same realm.

3. **Pick what in the card names the person** - `X509_IDENTITY` in the settings file:

X509_IDENTITY	Read from the certificate	The user name in AAP	When
`cn` (default)	the subject's common name	`doe.john.q.1234567890`	people are added by a list or by hand. Readable in AAP. It changes when a person's name changes.
`edipi`	the 10 digits at the end of the common name	`1234567890`	you want a name that never changes for a person. Less readable in AAP.

X509_IDENTITY	Read from the certificate	The user name in AAP	When
`upn`	the *Principal Name* in the subject alternative name	`1234567890123456@mil`	**the people come from Active Directory** (`LDAP_URL`): it is AD's `userPrincipalName`, the value smart card logon uses. Only the card's *Authentication* certificate has it: people must pick that one.
`email`	the email in the subject alternative name	the person's Keycloak user name	only the card's email certificates have it. An address changes: not the first choice.

The usual DoD choices: `upn` with Active Directory behind Keycloak, `cn` or `edipi` without. Look at how STIG Manager's realm names one person (**Users** in Keycloak's console) and pick the same, so a person has one name in both. Keycloak stores a user name in small letters: AAP shows `doe.john.q.1234567890`.

With `X509_USER_ATTRIBUTE=edipi` the value is looked up in a user attribute of that name, and the user name is free. `X509_IDENTITY=regex` and `X509_REGEX` read anything else from the subject.

4. **The admin password, in a file only you can read.** It is never typed on a command line:

```shell
install -m 600 /dev/null ~/keycloak-admin.pw
nano ~/keycloak-admin.pw
```

Put `KEYCLOAK_ADMIN_PASSWORD_FILE=/home/you/keycloak-admin.pw` in the settings file. (No such line: the script asks at the prompt.)

5. **The dry run.** Nothing is changed:

```shell
tools/aap-keycloak-realm.py --settings aap-keycloak.env
```

You should see `DRY RUN: nothing is changed`, then one line per thing:

```
CREATE  realm aap
CREATE  flow browser-x509: a copy of the browser flow
CREATE  flow browser-x509: the step X509/Validate Username Form, before the password forms,
  ALTERNATIVE
CREATE  flow browser-x509: the card names the person by the subject's common name
  (DOE.JOHN.Q.1234567890), as the user name
CHANGE  flow browser-x509: the password forms DISABLED
CHANGE  realm aap: its browser flow is browser-x509
CREATE  client aap (confidential, standard flow only, redirect URI
  https://aap.yoursite.mil/api/gateway/social/complete/*)
CREATE  client aap: the mapper groups
CREATE  client aap: the mapper audience
ok      client aap: preferred_username and email come from the realm's profile and email scopes
CREATE  group aap-admins
CREATE  group aap-operators
CREATE  group aap-users

People: none are made. Stock Keycloak does not make a user at a first card login: a person must be
  in the
  realm first. Name a list (AAP_USERS_FILE), add them in Keycloak's console, or set a federation
    (LDAP_URL).
AAP's authenticator (Generic OIDC) - its fields:
  OIDC Provider URL          https://keycloak.yoursite.mil/realms/aap
  OIDC Key                   aap
  OIDC Secret                the client's secret (--secret-file)
  OIDC JWT Algorithm(s)      RS256
  Username Key               preferred_username
  Groups Claim               groups
  the callback, in Keycloak  https://aap.yoursite.mil/api/gateway/social/complete/*
12 changes would be made. Nothing was changed (dry run). Run again with --apply to make them.
Requests: 1, all to /admin/realms/aap; to another realm: 0.
```

Read the lines. The realm's name is `aap`, not STIG Manager's. (The script stops by itself on a realm named in `KEYCLOAK_PROTECTED_REALMS`, and on a realm of this name that it did not make.)

- The lines under `AAP's authenticator` are what you type in AAP in step 5.3.
- A realm `aap` is there already, made by hand: you see `found` and `differs` lines. Go to "If you already set it up by hand" below.

6. **Make it:**

```shell
tools/aap-keycloak-realm.py --settings aap-keycloak.env --apply --secret-file ~/aap-client.secret
```

You should see the same lines, then `12 changes made in the realm aap.` and `to another realm: 0`. The client's secret is in `~/aap-client.secret` (mode 600), for step 5.3. It is not printed.

7. **Run it once more:**

```shell
tools/aap-keycloak-realm.py --settings aap-keycloak.env --apply
```

You should see only `ok` lines and `Nothing to change: the realm aap is as the settings say.`

8. **The people.** A good card is not enough: **the person must be in the realm** `aap`. Stock Keycloak has no step that makes a user at a first card login (this was tried: the login ends at *Invalid username or password*). So the script makes nobody by default, and says so. Three ways to put people in:

Way	Setting	What happens
Active Directory	`LDAP_URL`, `LDAP_GROUP_DN` and the lines beside them	a **read-only** federation. Only the members of one group get in, and nothing is kept by hand in Keycloak. The usual way: see "People from Active Directory" below.
A list	`AAP_USERS_FILE`	the script makes each person and puts them in their groups. One line each: `username,email,first name,last name,groups` (groups with `;`). The user name is what step 4.3 reads from the card.
By hand	-	Keycloak's console, realm `aap`: **Users > Add user**, then the **Groups** tab

A list, for example:

```
DOE.JOHN.Q.1234567890,john.doe@yoursite.mil,John,Doe,aap-admins;aap-users
ROE.JANE.A.1234567891,jane.roe@yoursite.mil,Jane,Roe,aap-operators
```

Run steps 4.5 and 4.6 again after a change to the list. You should see one `CREATE person` line per new person. The script never removes a person or takes one out of a group: do that in the console.

9. **The other card settings** - all off by default, each a line in the settings file, then steps 4.5 and 4.6 again:

Setting	on means
`X509_OCSP`	Keycloak asks the certificate's OCSP responder at every login (`X509_OCSP_RESPONDER` names another one). A certificate that names no responder is then refused. Keycloak must reach the responder.
`X509_CRL`	Keycloak reads the CRL the certificate names
`X509_PASSWORD_FORM`	Keycloak's user name and password form stays, for a person with no card. Off: the card is the only way in.
`X509_CONFIRM_PAGE`	Keycloak shows its "continue as ..." page after the card

nginx already checks each card against your CAs (`ssl_client_certificate`) before Keycloak sees it.

People from Active Directory (the MIM group)

Your identity manager (MIM) already decides who is in one Active Directory group. That group is the list of AAP's people. Keycloak reads it. Nobody is added or removed by hand in Keycloak.

- Keycloak **only reads** the directory (`READ_ONLY`). It never writes to it.
- **Only the members of `LDAP_GROUP_DN` get in.** A member of a group inside it counts too (`LDAP_GROUP_NESTED=off`: direct members only).
- The card finds the person: the card's *Principal Name* is compared with the account's `userPrincipalName`.
- The directory groups you name in `LDAP_TEAM_GROUPS` arrive in AAP as the token's `groups`, for the maps.
- A new member needs nothing done: Keycloak takes the person in at their first card login.

1. **Ask the directory's admins for five things.**

Ask for	It goes in	For example
a **read-only bind account**: its DN and its password. It reads the people and the groups, nothing more.	`LDAP_BIND_DN`, `LDAP_BIND_PASSWORD_FILE`	`CN=svc-keycloak,OU=Service,DC=corp,DC=example,DC=mil`
the **DN of the access group** (the one MIM fills)	`LDAP_GROUP_DN`	`CN=AAP-Access,OU=Groups,DC=corp,DC=example,DC=mil`
the DNs of the groups that are AAP's teams and admins	`LDAP_TEAM_GROUPS`, with `;` between them	`CN=AAP-Admins,OU=Groups,DC=corp,DC=example,DC=mil`
the server's name and port, and where the people are	`LDAP_URL`, `LDAP_USERS_DN`	`ldaps://dc01.corp.example.mil:636`, `OU=People,DC=corp,DC=example,DC=mil`
the **CA chain** that signed the server's certificate, as a PEM file	`LDAP_CA_FILE`, and Keycloak's trust store (step 3)	`/home/you/ad-ca-chain.pem`

2. **Look at one card and one account.** The two values must be the same text.
   - The card: in the browser's or Windows' certificate window, the *Authentication* certificate > **Details** > **Subject Alternative Name**. Read the line *Principal Name*, for example `1234567890123456@mil`.
   - The account: on a Windows machine of the domain, logged in as that person:

```
whoami /upn
```

What you see	What to set
the same text on both	nothing: `userPrincipalName` is the default
the account's `userPrincipalName` is something else, and the directory keeps the card's value in another attribute	`LDAP_CARD_ATTRIBUTE=<that attribute>` (the directory's admins know its name)
no attribute holds the card's value, and the account's `mail` is the address in the card	`X509_IDENTITY=email`: the card's email is compared with `mail`. People then pick an email certificate of the card.

`altSecurityIdentities` cannot be used: it holds the certificate in another form (`X509:<I>...<S>...`), not the Principal Name.

3. **Keycloak must trust the directory's CA - a change you make by hand.** Skip it when another realm of this Keycloak already reads the same directory. It is the `--truststore-paths` start option of step 4.1, with the chain's file added after a comma:

```
--truststore-paths=/opt/keycloak/conf/card-cas.pem,/opt/keycloak/conf/ad-ca-chain.pem
```

Mount the file into the Keycloak container. This **restarts Keycloak**: do it in a quiet hour. Without it the script's apply says `WARNING Keycloak could not bind`.

4. **The bind account's password, in a file only you can read:**

```shell
install -m 600 /dev/null ~/ldap-bind.pw
nano ~/ldap-bind.pw
```

5. **The settings.** In the settings file:

```
LDAP_URL=ldaps://dc01.corp.example.mil:636
LDAP_CA_FILE=/home/you/ad-ca-chain.pem
LDAP_USERS_DN=OU=People,DC=corp,DC=example,DC=mil
LDAP_BIND_DN=CN=svc-keycloak,OU=Service,DC=corp,DC=example,DC=mil
LDAP_BIND_PASSWORD_FILE=/home/you/ldap-bind.pw
LDAP_GROUP_DN=CN=AAP-Access,OU=Groups,DC=corp,DC=example,DC=mil
LDAP_TEAM_GROUPS=CN=AAP-Admins,OU=Groups,DC=corp,DC=example,DC=mil;CN=AAP-Operators,OU=Groups,DC=c
  orp,DC=example,DC=mil
```

   - Leave `X509_IDENTITY` and `AAP_GROUPS` without a line. With `LDAP_URL` the card is then read by its Principal Name (`upn`), and no group is made in Keycloak: the groups come from the directory.
   - `ldap://` without encryption is refused. A server with no `ldaps` port: `LDAP_STARTTLS=on` with `ldap://`.
   - Keycloak reads the changed people every 15 minutes and all of them once a day (`LDAP_SYNC_CHANGED_SECONDS=900`, `LDAP_SYNC_FULL_SECONDS=86400`).

6. **The dry run** (step 4.5). You should see these lines among the others:

```
CREATE  user federation directory: ldaps://dc01.corp.example.mil:636, READ_ONLY, the user name
  from sAMAccountName
        lets in only the members of CN=AAP-Access,OU=Groups,DC=corp,DC=example,DC=mil (nested
          groups count)
        the changed people are read every 900 s, all of them every 86400 s
CREATE  user federation directory: the LDAP attribute userPrincipalName is read into the user
  attribute aap-upn (what the card is looked up in)
CREATE  user federation directory: the groups AAP-Admins, AAP-Operators become Keycloak groups,
  with their members (the mapper aap-teams, read-only, nested members count)

People: they come from the directory, the members of AAP-Access. Nobody is made here.
  A member is taken in at the first card login, or by a sync. Out of the group: refused at the
    next login.
```

A `WARNING LDAP_GROUP_DN is not set` line means: everybody under `LDAP_USERS_DN` could log in. Name the group.

7. **Make it**, with one full read of the directory at the end:

```shell
tools/aap-keycloak-realm.py --settings aap-keycloak.env --apply --sync --secret-file
  ~/aap-client.secret
```

You should see `note Keycloak reached ldaps://dc01.corp.example.mil:636 and the bind account was accepted` and a `SYNC a full sync of the directory:` line with the number of people added and `0 failed`.

8. **Test it.**
   - A member of the group logs in with their card (step 6.2). In AAP the user name is the account's logon name (`sAMAccountName`), and the teams are the ones the maps give for the person's groups.
   - A person who is **not** in the group tries. They should see Keycloak's *Invalid username or password* page.
   - The check script's `LDAP sync` and `Groups` lines (step 6.5).

When a person is taken out of the group - what Keycloak 26 does. This was tried.

When	What happens
at the person's next card login	**refused**, at once. Keycloak asks the directory at every login, with the group in the question. No sync is needed for it.
a login session that is open in Keycloak	it lasts until it ends (the realm's *SSO Session Idle* and *SSO Session Max*: 30 minutes and 10 hours unless you changed them). To end it now, in Keycloak's console, realm `aap`: **Users >** the person **> Sessions > Logout all sessions**.
a session that is open in AAP	it lasts until AAP ends it or the person logs out. AAP asks Keycloak only at a login.
Keycloak's own copy of the person	it goes when Keycloak next looks that person up: their next login try, or an admin opening them under **Users**. The timed sync does not take it away (its result said `0 removed`). The copy lets nobody in.
an account disabled in the directory	refused at the next card login, the same way

- **To clear every copy by hand:** Keycloak's console, realm `aap`: **User federation >** the federation **> Action > Remove imported**. The directory is not touched. Each member is taken in again at their next login.
- **A person added to the group** gets in at their next card login. There is nothing to wait for.
- **A person moved to another team group** has the new teams in AAP at their next login to AAP.

If you already set it up by hand

The realm, its directory federation, its mappers and the client are there already, made in Keycloak's console. The script takes them as they are. It adds what is missing and lists what differs.

- **It finds things by what they are, not by their name:** the realm's LDAP federation, its group mapper, the mapper that reads `userPrincipalName` into a user attribute, the client's *Group Membership* mapper, and a browser flow that already has the card step. Each is used under the name you gave it (the `found` lines).
- **Missing:** it is made (`CREATE`). **There, another way:** a `differs` line, and it is left as it is.
- **Nothing is deleted, and nothing is renamed.** A mapper the script does not know stays.

1. **The settings name what you have.** `KEYCLOAK_REALM` is your realm's name, `KEYCLOAK_CLIENT_ID` your client's. The `LDAP_` lines say what your federation should be (the part above, step 5). No bind password is asked for: it is only needed when a federation is made.
2. **The dry run.** It only reads:

```shell
tools/aap-keycloak-realm.py --settings aap-keycloak.env
```

You should see lines like these:

```
note    the realm aap is there and this script did not make it. A dry run only reads it; --apply
  needs --adopt.
found   user federation corp-ad: the realm's LDAP federation. It is used under its own name
differs client aap: directAccessGrantsEnabled is true, settings say false
differs user federation corp-ad: changedSyncPeriod is -1, settings say 900
CREATE  client aap: the mapper groups (the groups, short names, as the claim "groups" in the ID
  token, the access token and userinfo)
```

3. **Read each `differs` line and decide.**

You want	Do
what is there	change the setting, so the line turns into `ok` at the next dry run. Or leave the line: it changes nothing.
what the settings say	step 5

4. **Add what is missing.** `--apply` on a realm the script did not make needs `--adopt`, which adds only the script's mark (one attribute of the realm) and changes nothing else:

```shell
tools/aap-keycloak-realm.py --settings aap-keycloak.env --apply --adopt
```

You should see `CREATE realm aap: the mark that this is AAP's realm (--adopt)`, one `CREATE` line per missing thing, and the `differs` lines again, untouched. `--adopt` is needed once. Without it the script stops with *this script did not make it. Nothing was changed.*

5. **Only if you want the `differs` fields changed:**

```shell
tools/aap-keycloak-realm.py --settings aap-keycloak.env --apply --update-existing
```

You should see one `CHANGE` line per thing, with each field as `old -> new`:

```
CHANGE  client aap: directAccessGrantsEnabled true -> false; redirectUris
  https://aap.yoursite.mil/api/gateway/social/complete/oidc/ ->
  https://aap.yoursite.mil/api/gateway/social/complete/*
```

Only the fields of the `differs` lines change. Run the dry run first, every time.

6. **The groups in the token.** A client made by hand often has no *Group Membership* mapper. Then the token has no `groups`, and AAP's maps give nobody a team. The script's `CREATE client aap: the mapper groups` line of step 4 adds it: the claim `groups` (`AAP_GROUPS_CLAIM`), full path off, in the ID token, the access token and userinfo. The check script's `Groups` line tells (step 6.5).
7. **AAP's method.** Compare the method you made in AAP with the lines the script prints under `AAP's authenticator (Generic OIDC) - its fields:` (step 5.3). Then do step 5.9: an LDAP method in AAP must be switched off.
8. **Run the check** (part 6) until nothing fails.

5. AAP: the login method and the maps

In AAP, as `admin`.

1. **Access Management > Authentication Methods > Create authentication.**
2. **Name:** `Keycloak`. **Authentication type:** `Generic OIDC`.
3. **Fill these fields.** Leave every other field as it is. The realm script prints the first six under `AAP's authenticator (Generic OIDC) - its fields:` (step 4.5), with your names in them.

Field	Value
OIDC Provider URL	`https://keycloak.yoursite.mil/realms/aap`

Field	Value
OIDC Key	`aap`
OIDC Secret	the text in `~/aap-client.secret` (step 4.6): `cat ~/aap-client.secret`, paste, then `rm ~/aap-client.secret`
Verify OIDC Provider Certificate	on
OIDC JWT Algorithm(s)	`RS256` (Red Hat: the login fails when this is left empty)
Username Key	`preferred_username`
Groups Claim	`groups` (the default is `Group`: it must be the realm script's `AAP_GROUPS_CLAIM`)
OIDC Callback URL	leave empty: AAP fills it when you save
Create objects	on: an organization or team a map names is made when it is missing
Enabled	on
Remove users	on: at each login a person leaves the teams their groups no longer give them

4. **Create Authentication Method.**
5. **Read the callback address.** Open the method you made. Under **Details**, copy the **OIDC Callback URL**. You should see an address that starts with `https://aap.yoursite.mil/api/gateway/social/complete/`. It starts with `http://`, or with another name: stop and see the table in part 6.
6. **Give Keycloak the exact address.** Until now the client accepts any address under `https://aap.yoursite.mil/api/gateway/social/complete/`. Put the address you copied in the settings file, exactly, with its last `/`:

```
AAP_REDIRECT_URI=https://aap.yoursite.mil/api/gateway/social/complete/<what AAP showed>/
```

Then run the realm script again (steps 4.5 and 4.6). You should see one line, `CHANGE client aap: redirectUris`.

7. **The maps.** In AAP, in the method, the **Mapping** tab > the **Add authentication mapping** list. Make these four. A group's name is typed exactly as the token has it: the names of `AAP_GROUPS`.

Map type	Name	Trigger	Groups	Other fields
Allow	`Only AAP people`	Group, operation *or*	`aap-admins`, `aap-operators`, `aap-users`	Revoke: on
Superuser	`AAP admins`	Group, operation *or*	`aap-admins`	Revoke: on
Organization	`Everyone in Default`	Always	-	Organization `Default`, Role *Organization Member*
Team	`Operators`	Group, operation *or*	`aap-operators`	Team `Operators`, Organization `Default`, Role *Team Member*, Revoke: on

- The *Allow* map with *Revoke: on* means: a person in none of the three groups cannot log in to AAP, even with a good card.
- On the other maps *Revoke: on* means: the role is taken away at the next login when the person has left the group.
- Maps run in the order of the list, the top one first.
- **With people from Active Directory** the groups are the directory's: type the names of `LDAP_TEAM_GROUPS` (`AAP-Admins`, `AAP-Operators`) in place of `aap-admins` and `aap-operators`. In the *Allow* map, name every one of them. The access group itself is not in the token: Keycloak already lets only its members in.

8. **Keep the local login.** In **Authentication Methods**, the **Local Database Authenticator** stays *Enabled*. Never switch it off: Red Hat says that locks out every local account, `admin` too.
   - The login page then shows the user name and password form **and** a **Keycloak** button.
   - `admin` logs in with the form. That is the way in when Keycloak is down.
   - nginx is down too: the gateway still answers at its own address, `https://aap01.yoursite.mil` (step 1.2).
   - Someone did switch it off: on the AAP host, as the install user - Red Hat's two commands, run inside the gateway's container *(the `podman exec` part is not in Red Hat's documents)*:

```shell
podman exec -it automation-gateway aap-gateway-manage authenticators --list
podman exec -it automation-gateway aap-gateway-manage authenticators --enable <ID>
```

9. **No password login beside the card.** Look at the other rows of **Authentication Methods**. An *LDAP*, *RADIUS* or *TACACS+* method that is *Enabled* lets a person in with a user name and password, past the card.
   - Open each one and switch *Enabled* off. Do not delete it: its maps are kept, and it is the way back.
   - The Local Database Authenticator is not one of them. It stays on (step 8).
   - The check script tells, when it has a token (step 6.4): `WARN AAP login the LDAP authenticator "..." is enabled next to the Keycloak one:` and under it *a password login that bypasses the card is on.*

If you pick the type "Keycloak" instead of "Generic OIDC"

Both work with Keycloak, and Red Hat prefers neither. This guide uses *Generic OIDC* because it reads Keycloak's keys by itself (nothing to paste again when the realm's key changes) and shows its callback address. The type *Keycloak* needs three things more:

1. **Keycloak Access Token URL:** `https://keycloak.yoursite.mil/realms/aap/protocol/openid-connect/token`
2. **Keycloak Provider URL:** `https://keycloak.yoursite.mil/realms/aap/protocol/openid-connect/auth`
3. **Keycloak Public Key:** in Keycloak, in the realm `aap`, **Realm settings > Keys** > the *RS256* row > **Public key**. Paste the text alone, without `-----BEGIN PUBLIC KEY-----` lines. When the realm's key changes, paste the new one.
4. **Keycloak OIDC Key** `aap`, **Keycloak OIDC Secret** the client secret, **Groups Claim** `groups` (older 2.5 builds do not have this field: name the mapper's claim `Group` there).
5. The *Audience* mapper this type needs is already there: the realm script makes it (`client aap: the mapper audience`). Without it the login fails with *Audience doesn't match*.

6. This type shows no callback address. Run the check script (part 6): its *Redirect* line prints the address AAP sends, for step 5.6.

6. Test

1. **Run the check.** From your workstation, in this repository's folder:

```shell
tools/aap-keycloak-check.sh --settings aap-keycloak.env
```

Your workstation does not trust your CA: set `CA_FILE` in the settings file. You should see only `PASS` lines, one `SKIP Token` line, and *Nothing failed.* One of the lines is `PASS Card realm aap refuses a caller with no card, and shows no password form`: the card flow is the realm's login. The script only reads: it sends GET requests and writes nothing. A `FAIL` line names the step of this guide to look at.

2. **Log in with a card.** Open a private window. Go to `https://aap.yoursite.mil`. Click **Keycloak**. Pick your card's certificate (with `X509_IDENTITY=upn`: the one named *Authentication*) and type the PIN. You should land in AAP, with your name at the top right. Then, as `admin`: **Access Management > Users >** that person - the **Teams** and **Organizations** tabs show what the maps gave. (What each map decided at the login: `https://aap.yoursite.mil/api/gateway/v1/users/<the user's id>/`.)
3. **A job's live output.** Launch any short job template. Open its **Output** while it runs. You should see the lines appear by themselves, without a reload of the page. That is the websocket through nginx.
4. **The API with a token.** As `admin`: **Access Management > Users > admin > Tokens > Create token** (scope *Read*). Put the token in a file only you can read, and run the check with it:

```shell
install -m 600 /dev/null ~/aap.token
nano ~/aap.token
AAP_TOKEN_FILE=~/aap.token tools/aap-keycloak-check.sh --settings aap-keycloak.env
rm ~/aap.token
```

You should see `PASS Token`. Then delete the token in AAP. The token is never typed on a command line: the script hands it to `curl` on standard input.

With the token the script also reads AAP's list of authentication methods. You should see `PASS AAP login no LDAP, RADIUS or TACACS+ method is enabled in AAP`. A `WARN`: step 5.9. (A token of a person who is no admin or auditor may not read the list: the line says `SKIP`.)

A person who logs in through Keycloak cannot make a token by default (*OAuth2 Tokens cannot be created by users associated with an external authentication provider*). Only if you want that: **Settings > Platform gateway > Allow external users to create OAuth2 tokens**.

5. **The card CAs, the directory and the groups.** Three more lines, each with a setting of its own:

Line	Needs	PASS means
`Card CAs`	`CARD_CA_FILE`: a copy of the file nginx's `ssl_client_certificate` line names	nginx asks a card from exactly the CAs of that file. The line names the file.
`LDAP sync`	`KEYCLOAK_ADMIN_PASSWORD_FILE` (step 4.4)	the realm's federation is on and read-only, it lets in one group only, and its last sync is not late

Line	Needs	PASS means
`Groups`	`KEYCLOAK_ADMIN_PASSWORD_FILE`	the tokens Keycloak would give a person carry the claim `groups`

```shell
tools/aap-keycloak-check.sh --settings aap-keycloak.env
```

- The `Groups` line looks at the first person of the realm. `CHECK_USER=jdoe` names another.
- No login is made for it and no token is issued: Keycloak's own *evaluate* is read.
- The admin password is read from the file and handed to `curl` on standard input. It is the one request of the script that is not a GET, and it changes nothing.
- Keycloak's admin address is closed from your workstation: run the script on the Keycloak host, with the `KEYCLOAK_ADMIN_URL` of step 4.2.
- `FAIL Groups a token of ... has no claim "groups"`: the client has no *Group Membership* mapper. Run the realm script (step 4.5): its `CREATE client aap: the mapper groups` line adds it.

When it fails

What you see	Why	What to do
`400 No required SSL certificate was sent` at `aap.yoursite.mil`	the AAP block asks for a card, or the request landed in STIG Manager's block	`ssl_verify_client off;` in the AAP block; `server_name` is the name typed; steps 2.7 to 2.9 again
STIG Manager's page shows at `aap.yoursite.mil` (or a 404 from its API)	nginx has no block for the name, so the first block answered	step 2.7 prints `0`: edit in place; then `nginx -t` and the reload
`502 Bad Gateway`	nginx does not reach the gateway	step 2.3 from the STIG Manager host; the address and port in `proxy_pass`; a firewall on the AAP host
The browser warns about the certificate	the certificate does not name `aap.yoursite.mil`, or its chain is missing	step 2.4: the file has the certificate **and** its chain
Keycloak: *Invalid parameter: redirect_uri*	the address AAP sent is not the one in the client	the address in the browser's address bar holds `redirect_uri=`. It starts with `https://aap.yoursite.mil/`: put it in `AAP_REDIRECT_URI` (step 5.6). It does not: the next two rows
The `redirect_uri` starts with `http://`	AAP thinks the browser used http	the block needs `proxy_set_header X-Forwarded-Proto https;`. Still http: set `gateway_main_url` (step 3.1) and run the installer again. Still http *(not in Red Hat's documents)*: add `{"setting": "REDIRECT_IS_HTTPS", "value": True}` to `gateway_extra_settings`
The `redirect_uri` holds `aap01.yoursite.mil` or has lost its port	nginx does not pass the name people typed	the block needs `proxy_set_header Host $http_host;` (not `$host`, not `$proxy_host`)
CSRF verification failed or *Origin checking failed* at the local login	AAP does not trust the new address	step 3.3
After the card: back at AAP's login page with an error, and nothing in AAP's user list	AAP cannot ask Keycloak for the token: no route, no trust, or Keycloak asks callers for a card	step 1.4 from the AAP host; `ssl_verify_client optional` on Keycloak's address (step 2.6); `custom_ca_cert` (step 3.1); on the AAP host `journalctl CONTAINER_NAME=automation-gateway --since -10m` shows the reason
The same, and the log says *Audience doesn't match*	the type *Keycloak* without the audience mapper	run the realm script again (step 4.5): it makes the mapper
The same, and the log names an algorithm or a signature	*OIDC JWT Algorithm(s)* is empty or not the realm's	`RS256` in step 5.3

What you see	Why	What to do
The person is in, but in no team, or not an admin	the groups did not arrive, or their names differ	*Groups Claim* is `groups` (step 5.3); the person is in the group in the realm `aap` (step 4.8) - with Active Directory: the group is in `LDAP_TEAM_GROUPS`; the map's group is spelled as in Keycloak; the check's `Groups` line (step 6.5)
AAP refuses a person after a good card login at Keycloak	the *Allow* map: the person is in none of its groups	put the person in a group in the realm `aap` (step 4.8), or widen the map (step 5.7)
Keycloak asks for a user name and password, not the card	the card flow is not the realm's login, or `X509_PASSWORD_FORM=on` and no card arrived	the check's `Card` line; the realm script again (step 4.5): it must end with *Nothing to change*
Keycloak: *We are sorry... Invalid username or password.* and the browser never asked for a card	nginx did not ask for one at Keycloak's address	the `server` block of Keycloak's address needs `ssl_client_certificate` and `ssl_verify_client optional;` (step 2.6). A browser that was refused once remembers it: use a new private window
The same page, after you picked the card	Keycloak's log says which of three (on the Keycloak host: `sudo podman logs --since 10m <keycloak> 2>&1 | grep -E 'LOGIN_ERROR|ssl-client-cert'`)	the next three rows
... the log says *HTTP header "ssl-client-cert" is empty*	nginx does not hand the card to Keycloak	block B's line in the Keycloak location (step 2.6), then steps 2.8 and 2.9
... the log says `error="invalid_user_credentials"` with no `x509_cert_subject` in the line	Keycloak does not read the header	the start options (step 4.1)
... the log says `error="user_not_found"` and names the card's subject	the card is good, the person is not in the realm `aap` - or under another name than the card gives. With Active Directory: the person is not in the access group.	step 4.8; compare the user name with what `X509_IDENTITY` reads (step 4.3)
The realm script: *HTTP 409 ... User exists with same email*	two lines of `AAP_USERS_FILE` have one address, or the person is there under another user name	one address per person; look under **Users** in the realm `aap`
The realm script stops: *this script did not make it*	a realm of that name was there before	pick another `KEYCLOAK_REALM`. When that realm is really AAP's: `--apply --adopt` ("If you already set it up by hand", step 4)

What you see	Why	What to do
The realm script prints `differs` lines	something in the realm is not as the settings say. It was left as it is.	"If you already set it up by hand", step 3
The realm script: `WARNING Keycloak could not bind`	Keycloak does not reach the directory, does not trust its certificate, or the bind account is refused	"People from Active Directory", step 3 (the trust store) and step 1 (the account); a firewall between Keycloak and the directory
A member of the access group is refused: `error="user_not_found"`	the card's Principal Name is not the account's `userPrincipalName`, or the person is in the group only through a group inside it and `LDAP_GROUP_NESTED=off`	"People from Active Directory", step 2
A person logs in to AAP with a password, no card	an LDAP, RADIUS or TACACS+ method is enabled in AAP	step 5.9
The check: `FAIL Groups`	the client has no *Group Membership* mapper	step 6.5
The check: `WARN Card CAs ... do not name the same CAs`	nginx asks cards of another chain than `CARD_CA_FILE` holds: a chain of another network is in one of the two	the `ssl_client_certificate` file of Keycloak's address (step 2.6); step 4.1's trust store
`403` or `404` at `https://stigman.yoursite.mil/reports/`	the container does not see the folder, or cannot read it	step 2.5: the `ls` check; the `,z` note; the folder and its files are readable by others (`chmod -R o+rX`)
A job's output shows only after a reload; the browser's console shows a failed `wss://`	the websocket headers do not reach AAP	the `map` block and the `Upgrade` / `Connection` lines; `proxy_http_version 1.1;` (block A, step 2.6)
A running job's output stops after about a minute	nginx's default 60 second read time	`proxy_read_timeout 600s;` in the block
413 Request Entity Too Large on an upload	nginx's default body size, or the gateway's own (5m)	`client_max_body_size` in the block; the gateway's: `gateway_nginx_client_max_body_size` in the inventory
A script with a token gets *401*	the token is old, or of a Keycloak user	make a new token as a local user, or allow tokens for external users (step 6.4)
After step 3.2 AAP's own pages or jobs fail	AAP's parts do not reach `aap.yoursite.mil` from the AAP host	step 1.5 from the AAP host; the CA in `custom_ca_cert`; or take `gateway_main_url` out again (part 7)
Event-Driven Ansible's event streams stop	they use their own mTLS, which a proxy that ends TLS cannot pass	point `eda_event_stream_url` at the AAP host itself, not at nginx *(not in Red Hat's documents)*

7. Roll back

Each step undoes one part. Do them in this order. `admin` can log in the whole time.

1. **AAP: switch the Keycloak login off. Access Management > Authentication Methods > Keycloak** - the *Enabled* switch off. The login page shows only the form again. (The Local Database Authenticator stays on.)
2. **AAP: the old address.** In the installer's `inventory`, remove the `gateway_main_url` line (and a `gateway_extra_settings` line you added). Run the installer again (step 3.2). Remove `https://aap.yoursite.mil` from **Csrf trusted origins** (step 3.3). Do this before step 3: until then AAP's own parts use the name.
3. **nginx: take the blocks out.** On the STIG Manager host, edit the file in place and delete everything between each `begin` line and its `end` line (AAP, Keycloak's card, reports). You changed `ssl_verify_client` to `optional` in step 2.6: it may stay, or go back to `on`. (Nothing else in the file changed since step 2.2? Then `sudo cp <the file>.before-aap <the file>` does all of it.) Then:

```shell
sudo podman exec <nginx> nginx -t
sudo podman exec <nginx> nginx -s reload
```

You should see *test is successful*, and STIG Manager works as before.

4. **Keycloak: the realm.** The script never deletes. In Keycloak's console, pick the realm **aap** at the top left - read the name twice - then **Realm settings**: the *Enabled* switch off. Nobody can log in through it, and everything in it is kept. To remove it for good: **Realm settings > Action > Delete**. The `aap-realm` client in `master` goes with it. STIG Manager's realm is another realm: leave it alone. The start options of step 4.1 may stay: STIG Manager's own card login uses them.
5. **The report website.** It went with block C. The `Volume=` line of step 2.5 may stay; taking it out is one more restart of nginx. Set `report_web_url` back in the report jobs.
6. **The name.** Ask for the DNS record `aap.yoursite.mil` to be removed. Remove the two `aap-*.pem` files from the certificate folder.

People open AAP at its own address again, with the local login.

Building it in an air-gapped enclave

The same build on another network: a classified one, or one with no way out. Three things are different there.

- **Another PKI.** That network's tokens chain to other CAs than the cards of the first network. nginx's `ssl_client_certificate` file and Keycloak's `--truststore-paths` take **that network's chain**. Never copy the first network's chain over.
- **The Principal Name can be written another way** on that network's tokens. Look at one token and one directory account before anything else (step 3 below).
- **No internet, and none is needed.** The three scripts call only the addresses in the settings file. A proxy in the environment is not used. They need Python 3.6, `bash`, `curl` and `openssl` - all in RHEL - and nothing to install. No image is downloaded: nginx and Keycloak are the containers that network's STIG Manager already runs.

Every address, DN, CA file and realm name comes from the settings file. Nothing of a network is written in a script. **One settings file per network**, and it never goes into Git.

1. **Bring this, and only this:**

Bring	Do not bring
the repository's release zip (`docs/`, `tools/`)	your settings file `aap-keycloak.env`
optional: a realm export of the realm that works, for step 10 (its secrets are masked)	any password file, `~/aap-client.secret`, an AAP token
	the first network's CA chains and certificate keys

The client's secret is **made new** in the enclave (step 9). No secret crosses.

The export, on the first network: Keycloak's console, realm `aap`: **Realm settings > Action > Partial export**, both switches on, **Export**. It holds that network's host names and DNs: your transfer rules say whether it may cross. Open the file and search for `secret`: every value must read `**********`.

2. **Collect in the enclave.** Ask for each line:

What	From whom	It goes in
the names: AAP, the AAP host, Keycloak, STIG Manager	DNS (step 1.5)	`AAP_URL`, `AAP_UPSTREAM`, `KEYCLOAK_URL`, `STIGMAN_URL`
a server certificate for the AAP name, and the chain that signed it	that network's PKI (step 1.6)	the certificate folder; `CA_FILE`
the CA chain of that network's **tokens**	that network's PKI	nginx's `ssl_client_certificate` file, Keycloak's `--truststore-paths`, `CARD_CA_FILE`
the directory: server, bind account, access group, team groups, its CA chain	the directory's admins ("People from Active Directory", step 1)	the `LDAP_` lines
STIG Manager's realm there	Keycloak's console	`KEYCLOAK_PROTECTED_REALMS`

3. **Look at one token and one account** ("People from Active Directory", step 2). Set `LDAP_CARD_ATTRIBUTE` or `X509_IDENTITY` when they do not match as they are.
4. **The settings file**, new, from the example:

```shell
cp tools/aap-keycloak.env.example aap-keycloak.enclave-a.env
nano aap-keycloak.enclave-a.env
```

Its last part, *ENCLAVE*, lists every line that is this network's own. Use this file's name in place of `aap-keycloak.env` in every command of this guide.

5. **Part 1**, the checks. Nothing is changed.
6. **Part 2**, the nginx blocks:

```shell
tools/aap-nginx-blocks.py --settings aap-keycloak.enclave-a.env
```

Before the reload, read the `ssl_client_certificate` line of the `server` block of Keycloak's address. The file it names must hold the chain of this network's tokens.

7. **Step 4.1**, Keycloak's start options. `--truststore-paths` names this network's token chain, and the directory's CA chain.
8. **Part 3**, AAP behind the proxy.
9. **The realm.** The dry run, then the apply. The apply makes a new client secret, here:

```shell
tools/aap-keycloak-realm.py --settings aap-keycloak.enclave-a.env
tools/aap-keycloak-realm.py --settings aap-keycloak.enclave-a.env --apply --sync --secret-file
  ~/aap-client.secret
```

The enclave's realm was built by hand already: "If you already set it up by hand".

10. **Compare with the realm that works** - optional, with the export of step 1:

```shell
tools/aap-keycloak-realm.py --settings aap-keycloak.enclave-a.env --compare realm-export.json
```

It only reads. Nothing is changed, and nothing of the export is ever imported. Secrets and ids are not compared. You should see one `differs` line per field that is not the same, then a count:

```
differs client aap: redirectUris is
  https://aap.enclave-a.yoursite.mil/api/gateway/social/complete/* here, the export says
  https://aap.yoursite.mil/api/gateway/social/complete/*
differs user federation directory: connectionUrl is ldaps://dc01.enclave-a.example.mil:636 here,
  the export says ldaps://dc01.corp.example.mil:636

2 differences; 9 things the same on both sides.
```

Addresses, DNs and host names differ between two networks on purpose. Every other line is something to line up: change a setting and run step 9 again, or decide the difference is wanted.

11. **AAP's method.** The realm script prints the fields under `AAP's authenticator (Generic OIDC) - its fields:`. Type them in step 5.3, with the secret of `~/aap-client.secret`. Then the maps (step 5.7) and step 5.9.
12. **Check, until every line is PASS:**

```shell
tools/aap-keycloak-check.sh --settings aap-keycloak.enclave-a.env
```

The `Card CAs` line names the file it used: `PASS Card CAs nginx on keycloak.enclave-a.yoursite.mil takes a card from the 2 CAs of CARD_CA_FILE=/home/you/card-ca-chain.pem`. A `WARN` there lists the CAs nginx asks for that are not in the file: the wrong chain is in one of the two. Then steps 6.4 and 6.5 for the `AAP login`, `LDAP sync` and `Groups` lines.

13. **Log in with a token of that network** (step 6.2).

The enclave has no STIG Manager yet, so no nginx and no Keycloak container: set STIG Manager up there first. Its images are carried over as files, as docs/EXECUTION_ENVIRONMENT.md does for an execution environment. On a machine that has them:

```shell
sudo podman images --format '{{.Repository}}:{{.Tag}}' | grep -i -E 'nginx|keycloak|auth'
sudo podman save -o keycloak.tar <the Keycloak image>
sudo podman save -o nginx.tar <the nginx image>
sha256sum keycloak.tar nginx.tar
```

In the enclave, on the STIG Manager host. Compare the two sums first:

```shell
sha256sum keycloak.tar nginx.tar
sudo podman load -i keycloak.tar
sudo podman load -i nginx.tar
sudo podman tag <the Keycloak image> <the name the compose file or unit uses>
sudo podman tag <the nginx image> <the name the compose file or unit uses>
```

You should see both under `sudo podman images`. The containers then start from the local images: nothing is pulled.

The three scripts

All three read the same settings file with `--settings` (`tools/aap-keycloak.env.example`). The environment wins over the file. A setting of another script in the file is passed over. `--help` on each prints its full list.

tools/aap-nginx-blocks.py

It writes `out/aap-nginx-blocks.conf` and `out/aap-nginx-steps.txt` (`--out` names another folder, `--print` only prints). It never reads or writes nginx's file, never calls podman or nginx, and refuses a folder under `/etc` or one named `nginx`.

Setting	Meaning
`AAP_URL`, `AAP_UPSTREAM`	the name people type; the gateway itself (step 1.2). Required.
`KEYCLOAK_URL`	Keycloak's address. Required.
`NGINX_CONTAINER`, `NGINX_CONF`	the container's name (default `stigman-nginx`) and its file as the container sees it: the steps name them
`NGINX_LISTEN`	the `listen` line of STIG Manager's block, default `443 ssl`
`NGINX_TLS_DIR`, `AAP_TLS_CERT`, `AAP_TLS_KEY`	the certificate folder in the container and AAP's two files
`KEYCLOAK_NGINX`	`lines` (default), `server` or `off`
`KEYCLOAK_UPSTREAM`, `KEYCLOAK_TLS_CERT`, `KEYCLOAK_TLS_KEY`, `NGINX_CLIENT_CA`, `NGINX_VERIFY_DEPTH`	for `KEYCLOAK_NGINX=server`: Keycloak itself, its certificate, the CAs of people's cards
`KEYCLOAK_VERIFY_HEADER`	`on`: also pass nginx's verdict (`ssl-client-verify`), for a Keycloak started with `trust-proxy-verification`
`REPORT_WEB_FOLDER`	the report website's folder on the STIG Manager host. Empty: no block C.
`REPORTS`, `REPORTS_PATH`	`location` (default) under `/reports/`; or `server` with `REPORTS_URL`, `REPORTS_TLS_CERT`, `REPORTS_TLS_KEY` for a name of its own (it asks for the card as STIG Manager does)

Setting	Meaning
REPORTS_MOUNT	where the folder shows in the container, default /usr/share/nginx/site-reports
REPORTS_UPSTREAM	no restart: a web server that already serves the folder
STIGMAN_URL	STIG Manager's address, for the report website's address in the steps

tools/aap-keycloak-realm.py

Dry run by default. It makes what is missing. What is there is left as it is, and a difference is one `differs` line. It never deletes and never renames. It calls Keycloak only (and, for `LDAP_CA_FILE`, your directory): no proxy is used.

Option	Meaning
--apply	make the changes
--via rest, --via kcadm	pick the way (step 4.2)
--secret-file FILE	with --apply: write the client's secret to FILE (mode 600)
--adopt	with --apply: work in a realm of this name that the script did not make. It adds one attribute to the realm, its mark.
--update-existing	change the fields the differs lines name, and no others
--sync	with --apply and LDAP_URL: one full sync of the directory at the end, and its result
--compare FILE	print how the realm differs from a realm export of another Keycloak. It only reads: nothing is imported.

Exit code 0 done, 1 Keycloak refused or could not be reached, 2 a setting is missing or wrong.

Setting	Meaning
KEYCLOAK_URL, AAP_URL	required
KEYCLOAK_REALM	the new realm, default aap
KEYCLOAK_PROTECTED_REALMS	realms it refuses to work in, default master. Add STIG Manager's.
KEYCLOAK_ADMIN_URL, KEYCLOAK_ADMIN_REALM, KEYCLOAK_ADMIN_USER, KEYCLOAK_ADMIN_PASSWORD_FILE, CA_FILE, KEYCLOAK_TIMEOUT	the admin login. The password comes from the file or the prompt, never from the command line.
KEYCLOAK_CONTAINER, KEYCLOAK_RUNTIME, KEYCLOAK_KCADM, KEYCLOAK_KCADM_SERVER	the kcadm way
KEYCLOAK_CLIENT_ID, AAP_REDIRECT_URI	the client (default aap) and its exact redirect URI (step 5.6)

Setting	Meaning
AAP_GROUPS, AAP_GROUPS_CLAIM	the groups to make (default aap-admins,aap-operators,aap-users; none with LDAP_URL) and their claim (default groups)
X509_FLOW, X509_IDENTITY, X509_REGEX, X509_USER_ATTRIBUTE	the card flow's name and what names the person (step 4.3)
X509_PASSWORD_FORM, X509_CONFIRM_PAGE, X509_OCSP, X509_OCSP_RESPONDER, X509_CRL	step 4.9
AAP_USERS_FILE	the list of people (step 4.8)
LDAP_URL, LDAP_USERS_DN, LDAP_BIND_DN, LDAP_BIND_PASSWORD_FILE	the read-only federation ("People from Active Directory"): the directory's ldaps:// address, where the accounts are, the account that reads, the file with its password
LDAP_STARTTLS, LDAP_CA_FILE	on for an ldap:// address with StartTLS (default off; plain ldap:// is refused); the directory's CA chain, for the script's own look at the directory's certificate
LDAP_GROUP_DN, LDAP_GROUP_NESTED	the one group whose members may log in; on (default): a member of a group inside it counts too (Active Directory only)
LDAP_TEAM_GROUPS, LDAP_TEAM_MAPPER	the directory groups that become Keycloak groups, DNs with ; between them; the name of the mapper made for them (default aap-teams)
LDAP_CARD_ATTRIBUTE, LDAP_USERNAME_ATTRIBUTE	the LDAP attribute that holds what the card gives (default userPrincipalName with X509_IDENTITY=upn); the one that becomes the user name
LDAP_SYNC_CHANGED_SECONDS, LDAP_SYNC_FULL_SECONDS	Keycloak's timed syncs: the changed people (default 900), all the people (default 86400). 0: never.
LDAP_NAME, LDAP_VENDOR	the federation's name in the realm (default directory); ad (default), rhds or other

What it makes in the realm, and nothing else:

Thing	How
the realm	enabled, no self-registration, marked as made by this script
the flow browser-x509	a copy of Keycloak's browser flow, with the step *X509/Validate Username Form* before the forms, and the forms disabled (X509_PASSWORD_FORM=on keeps them). It is set as the realm's browser flow.
the client aap	confidential, the standard flow only, the redirect URI under AAP_URL
the mappers	groups (short names) and the audience; preferred_username and email come from Keycloak's own profile and email scopes
the groups	AAP_GROUPS
people	only with AAP_USERS_FILE

Thing	How
the federation	only with `LDAP_URL`: read-only, with the filter of `LDAP_GROUP_DN`, the timed syncs, the mapper of `LDAP_CARD_ATTRIBUTE` and the group mapper of `LDAP_TEAM_GROUPS`

In a realm that was there before, each thing is found by what it is, not by its name: the LDAP federation, its group mapper, the client's *Group Membership* mapper, a browser flow that has the card step already.

tools/aap-keycloak-check.sh

Read-only. It never writes a file, never calls podman or nginx, and sends only GET requests. One request is not a GET: the admin login of the `LDAP sync` line, which changes nothing. It asks the addresses in its settings and nothing else. No proxy is used, whatever `http_proxy` says.

Setting	Meaning
`AAP_URL`	the address people type, https, no path. Required.
`KEYCLOAK_URL`	Keycloak's address without `/realms/...`. Required.
`KEYCLOAK_REALM`	default `aap`
`KEYCLOAK_CLIENT_ID`	default `aap`; only compared with what AAP sends
`CA_FILE`	your CA chain (PEM). Default: the machine's trust store.
`CARD_CA_FILE`	the CAs of people's cards (PEM), for the *Card CAs* line. No file: the CAs nginx asks for are printed and not compared.
`AAP_TOKEN_FILE`	a file with one AAP token, for the *Token* and *AAP login* lines
`KEYCLOAK_ADMIN_PASSWORD_FILE`	a file with the Keycloak admin's password, for the *LDAP sync* and *Groups* lines. No file: both are skipped. The script never asks at a prompt.
`KEYCLOAK_ADMIN_URL`, `KEYCLOAK_ADMIN_USER`, `KEYCLOAK_ADMIN_REALM`	the admin address when it is not `KEYCLOAK_URL`; the account and its realm (default `admin`, `master`)
`LDAP_URL`	when set, a realm with no federation is a FAIL of *LDAP sync* (else a SKIP)
`CHECK_USER`	the person whose tokens the *Groups* line looks at. Default: the first person of the realm.
`AAP_GROUPS_CLAIM`	the claim the *Groups* line looks for, default `groups`
`CHECK_TIMEOUT`	seconds per request, default 10

Line	PASS means
DNS	both names resolve on this machine
TLS	the AAP name's certificate is trusted and is for that name; a second line says until when

Line	PASS means
Gateway	AAP's gateway answers through nginx, and nginx asks for no card
Keycloak	the realm's `.well-known/openid-configuration` answers and names the same address
Redirect	AAP sends the login to this Keycloak; the way back is `https://` and the AAP name; Keycloak knows that redirect URI
Card	the realm refuses a caller with no card and shows no password form: the card flow is its login. A `WARN`: it shows a password form.
Websocket	the `Upgrade` headers reach AAP (HTTP 101). A `WARN` here: AAP wanted a login first - use step 6.3.
Card CAs	nginx asks a card from the CAs of `CARD_CA_FILE` on Keycloak's address. The line names the file. A `WARN`: the two lists differ, and the names follow.
Token	only with `AAP_TOKEN_FILE`: the API answers the token
AAP login	only with `AAP_TOKEN_FILE`: no LDAP, RADIUS or TACACS+ method is enabled in AAP beside the Keycloak one (step 5.9)
LDAP sync	only with `KEYCLOAK_ADMIN_PASSWORD_FILE`: the federation is on; a second line says when the last sync finished. A `WARN`: it is not read-only, it has no group filter, or the timed sync is late.
Groups	only with `KEYCLOAK_ADMIN_PASSWORD_FILE`: the tokens Keycloak would give a person carry the groups claim

It ends with exit code 0 when nothing failed, 1 when a check failed, 2 when a setting is missing.

Where each step comes from

Red Hat's documents are *Red Hat Ansible Automation Platform 2.5* and *2.6* at `https://docs.redhat.com`. Keycloak's is the *Server Administration Guide* on Keycloak's own site. nginx's is the documentation on nginx's own site.

Step	Document, section
1.2 the containers, the port	Red Hat, *Containerized installation*: the inventory variables (`envoy_https_port`), *Troubleshooting* (the container names)
2.6 block A: HTTP/1.1 to the gateway	Red Hat, *Containerized installation*, "Configuring a HAProxy load balancer": *HTTP/2 is not supported*
2.6 block A: the websocket lines	nginx, "WebSocket proxying"
2.6 block A: the other headers, the timeouts, the body size	nginx, the `ngx_http_proxy_module` and `ngx_http_core_module` pages. Red Hat has no nginx example.
2.6 `ssl_verify_client`, `$ssl_client_escaped_cert`	nginx, the `ngx_http_ssl_module` page
3.1, 3.2 `gateway_main_url`, `custom_ca_cert`, the installer	Red Hat, *Containerized installation*: "Configuring a HAProxy load balancer", the inventory variables, "Installing containerized Ansible Automation Platform"

Step	Document, section
3.3 Csrf trusted origins	Red Hat 2.6, *Access management and authentication*, "Configuring platform security"
3.4 Remote Host Headers, Proxy IP Allowed List	Red Hat, *Configuring automation execution*: "Enabling proxy support through a load balancer", "Configuring known proxies"
2.5 the mount	podman's documents: `podman-systemd.unit` (`Volume=`), `podman-run` (`--volume`, the `z` option)
2.6 block C	nginx, the `ngx_http_core_module` page: `alias`, `limit_except`
4.1 the start options	Keycloak, *Server Administration Guide*, "X.509 client certificate user authentication": "Enabling X.509 client certificate lookup" (the NGINX provider); *Server Configuration Guide*, "Using a reverse proxy"
4.3, 4.5 the card flow	Keycloak, *Server Administration Guide*, "X.509 client certificate user authentication": "Adding X.509 client certificate authentication to browser flows"
4.5 the realm, the client, the mappers, the groups	Keycloak, *Admin REST API* and *Server Administration Guide*: "Managing OpenID Connect clients", "OIDC token and SAML assertion mappings" (*Group Membership*, *Audience*), "Admin CLI" (`kcadm.sh`)
4.8, "People from Active Directory": the federation	Keycloak, *Server Administration Guide*, "Lightweight Directory Access Protocol (LDAP) and Active Directory": the storage mode (*READ_ONLY*), "Custom user LDAP filter", "Sync settings", "LDAP mappers" (*User Attribute*, *Group*), "Connect to LDAP over SSL"
"People from Active Directory": nested groups	Microsoft, *Search Filter Syntax*: the matching rule `1.2.840.113556.1.4.1941` (a member through a chain of groups)
"If you already set it up by hand", the enclave: the export	Keycloak, *Server Administration Guide*, "Exporting and importing realms" (*Partial export*: secrets are masked)
5.9 the other methods	Red Hat, *Access management and authentication*: "Configuring LDAP authentication", "Configuring RADIUS authentication", "Configuring TACACS+ authentication" (the *Enabled* field of each)
5.1 to 5.6 the method	Red Hat, *Access management and authentication*: "Configuring generic OIDC authentication", "Troubleshoot Generic OIDC Single Sign-On authentication failures"
5.7 the maps	Red Hat, the same guide: "Allow mapping", "Organization mapping", "Team mapping", "Superuser mapping", "Authenticator map types", "Authenticator map triggers", "Reviewing authenticator map results"

Step	Document, section
5.8 the local login	Red Hat, the same guide: "Enabling and disabling the local authenticator"
the type "Keycloak"	Red Hat, the same guide: "Configuring keycloak authentication" (and its *Audience doesn't match* note)
6.4 tokens	Red Hat, the same guide: "Enabling OAuth2 token creation for external users"

Not in Red Hat's documents

These parts of the guide are not written in Red Hat's documents. They come from reading the source code of AAP's login and web page, or from nginx's own documents. Each is marked in its step.

- **nginx in front of the gateway.** Red Hat describes HAProxy only. The nginx block is built from nginx's documents and Red Hat's two rules for a load balancer (HTTP/1.1, no TLS passthrough).
- **No sub-path.** No sentence says it. The web page's source asks for `/api/...` from the root.
- **The callback address** `https://<name>/api/gateway/social/complete/<id>/`: from the login's source code. Step 5.5 reads the real one from AAP, so the guide does not depend on it.
- `CSRF_TRUSTED_ORIGINS` **and** `REDIRECT_IS_HTTPS` **through** `gateway_extra_settings` on a containerized install. Red Hat documents `REDIRECT_IS_HTTPS` for the OpenShift install, and the CSRF field for 2.6's page only.
- `/api/gateway/v1/ping/` **and** `/api/gateway/v1/me/`, used by the checks: the web page uses them, the documents do not list them.
- `podman exec ... aap-gateway-manage authenticators`: Red Hat gives the command without saying where to run it on a containerized install.
- **Event streams behind a proxy that ends TLS.**
- **The websocket check without a login.** The script cannot log in with a card. When AAP answers 401 or 403 to its websocket request it says `WARN`, and step 6.3 is the real test.
- `/api/gateway/v1/authenticators/`, read by the check's `AAP login` line: the web page uses it. It was not tried against a real AAP.
- **Switching an LDAP method off beside the Keycloak one** (step 5.9). Red Hat documents each method's *Enabled* field. That a password method beside it is a way past the card is this guide's own conclusion.
- **The report website behind STIG Manager's nginx.** Not Red Hat's subject. Block C is nginx's own `alias`.

What was tried

On Keycloak **26.0.6** (the image STIG Manager's project publishes for its login) behind nginx (the `stable` image), with a test CA, an intermediate CA and test cards. The directory was a test domain controller (Samba's Active Directory, over `ldaps://`) with made-up people. No real name, card or directory was used.

Tried	Result
the realm script through `rest` and through `kcadm`	the dry run changes nothing; `--apply` makes the realm; a second `--apply` finds nothing to change

Tried	Result
another realm in the same Keycloak, compared before and after	unchanged. In `master`, Keycloak itself added the `aap-realm` client.
a card login through nginx to the realm, then the token	back at the redirect URI; the token has `preferred_username`, `email`, `groups` and the audience `aap`
`X509_IDENTITY` `cn`, `edipi`, `upn`, `email`, and `edipi` with `X509_USER_ATTRIBUTE`	each finds the person
a card of a person who is not in the realm; no card	Keycloak's *Invalid username or password* page (HTTP 400)
Keycloak without the start options of step 4.1	the same page: the card never reaches Keycloak
a made-up `ssl-client-cert` header	through nginx: refused. Straight to Keycloak's own port: **accepted** - see the warning in part 2.
`X509_OCSP=on` with a certificate that names no responder	refused
the generated blocks A, B (`lines` and `server`) and C (`location`, `server`, `REPORTS_UPSTREAM`)	`nginx -t` passes; the report folder answers with a card (200), without one (400), to a POST (403); the card login works through block B
the two Python scripts under Python 3.6	they run
the federation on a new realm: `--apply`, then `--apply` again	13 changes; then nothing to change
a member of the access group, never synced, logs in with a card	taken in at that login; the token has the directory groups of `LDAP_TEAM_GROUPS`
the same person taken out of the group	refused at the next card login. Keycloak's copy goes when the person is next looked up. A session that was open stays open until it ends.
a disabled account in the group	refused
a realm made by hand in the console (other names for the flow, the federation and the mappers)	the dry run prints `found` and `differs` lines; `--apply` alone stops; `--apply --adopt` makes only what is missing; `--update-existing` changes the named fields; nothing is deleted
`--compare` with the realm's own export; with a changed one	0 differences; the changed fields, one line each
the check script against that nginx and Keycloak, with a proxy set in the environment	the proxy is not used; `Card CAs`, `LDAP sync` and `Groups` say PASS; with another CA file `Card CAs` says WARN and names the CAs

Not tried:

- a real AAP. Parts 3, 5 and 6 come from Red Hat's documents and AAP's source code. The check's `AAP login` line ran against a made-up answer only.
- a real CAC;
- Microsoft's own Active Directory. The test domain controller was Samba's.

- `LDAP_STARTTLS=on` against a directory;
- Keycloak versions other than 26.0.6;
- STIG Manager's own nginx file at your site.